Written Document for ECM1414

1. Introduction

The student event planner problem provides us with an input of events, each with a name, duration, price and enjoyment value. The goal is to find the optimal combination of events to provide maximum enjoyment, while staying under defined limits for cost and time. We are also given the additional restriction of using each event once at most.

The coursework brief instructs us to find the optimal solution to this problem, for a range of different input sizes and limits for cost and time. Beyond this, we are also expected to compare the algorithmic efficiency of 2 different approaches: brute force, and dynamic programming.

2. Design and Approach

We decided to make each step of the process its own module, to allow for more flexibility and to make debugging easier, because each step of the process could be tested independently. Keeping the file interpreter and output modules separate from the algorithm code also saved us from writing specific interpreters and outputs for both. This helped to save time and resources.

The decision was made to use an array as our input and output for both algorithms, because of its ease to implement . For the Dynamic programming algorithms, we made use of a matrix (2D array), to implement the lookup table memoisation again due to its ease of implementation into . In both cases, we did not find that more complex data structures, such as using a tree structure for lookup, gave a significant performance boost to our algorithms.

For the Brute Force algorithm, we assumed that any 2 subsets with the same events in a different order would not have any difference in enjoyment value. Because of this, the amount of subsets that would need to be iterated through by the for loop was lowered. The checks with in this for loop by overwriting an array containing the current arrangement for each new iteration were simplified.

For dynamic programming, we assumed that the additional overhead of creating and filling a lookup table for results would still provide a sufficiently large speedup in the algorithm to be viable.

A binary was used in Brute force to simplify the creation of the subsets. This allowed us to reuse the same array memory address, instead of creating a unique variable for each subset, reducing overhead

3. Algorithms

File Reader Pseudocode

```
# FILE READING ALGORITHM

# activities will be used to store the list of loaded activities

activities = []


OPEN activities_file_location AS activity_file:

    # Read the first line of the file

READ LINE 1 OF activity_file AND STORE IN activities_count


    # Read the first line of the file

READ LINE 2 OF activity_file AND STORE IN duration_and_budget

# Seperate the maximum duration and maximum budget (separated by a space)

duration_and_budget = SPLIT duration_and_budget AT CHAR " "


    max_duration = duration_and_budget[0]

    budget = duration_and_budget[1]


    FOR i=0 TO i=activities_count:

        # Read a line of the file

    READ LINE (i + 3) OF activity_file AND STORE IN activity_data

    # Seperate the activity name, duration, cost and enjoyment

    activity_data = SPLIT activity_data AT CHARS " "
```

```
            # Use activity_data to create a new activity
            new_activity = {
            "name": activity_data[0],
                    "duration": activity_data[1],
                    "cost": activity_data[2],
                    "enjoyment": activity_data[3]
            }

            # Add the new activity to the activities list
            APPEND new_activity TO activities
        END FOR

        # Close the file
        CLOSE FILE file

# Return the max duration, budget and activities list
RETURN max_duration, budget, activities
```

Brute Force Pseudocode

```
#BRUTE FORCE ALGORITHM
activities = doc_array
best_arrangement = []
best_arrangement_enjoyment = 0
activities_length = LENGTH OF activities
# Calculate the number of possible combinations
arrangement_count = (2 TO THE POWER OF activities_length)
```

```
# Iterate through all possible arrangement combinations
FOR i=0 TO i=arrangement_count:
    # Uses .bin function
    binary_representation = GET BINARY EQUIVALENT OF i

    # Adds 0s to the beginning of binary number until it is the specified length
    WHILE LENGTH OF binary_representation < activities_length:
        ADD “0” TO THE FRONT OF binary_representation
    END WHILE

    # Used to store the current arrangement
    current_arrangement = []
    # List of all digits in binary representation of current arrangement
    bin_rep_list = []

    # Iterates through each digit and adds it to the binary representation list
    FOR number IN binary_representation:
        APPEND number TO bin_rep_list
    END FOR

    current_arrangement_joy = 0
    current_arrangment_duration = 0

    # Iterate through the binary list
    FOR item IN activities AND char IN bin_rep_list:
        # If current task binary representation positive (task should be included)
        IF char IS 1:
            # Adds that item to the current arrangement
```

```
                APPEND item TO current_arrangement
                ADD item["enjoyment"] TO current_arrangement_joy
                # Saves the arrangement's values into variables
                ADD item["duration"] TO current_arrangment_duration
            END IF
        END FOR

        # Checks that arrangement is within the maximum duration
        IF current_arrangment_duration < maximum_duration:
            # Checks if this arrangement has the highest joy
            IF current_arrangement_joy > best_arrangement_enjoyment:
                # Sets as the new best
                best_arrangement = current_arrangement
                best_arrangement_enjoyment = current_arrangement_joy
            END IF
        END IF
    END FOR
    # Best arrangement is returned
    RETURN best_arrangement
```

In a Brute force algorithm, all elements in a set are checked and the element (or subset of elements in this case) with the highest value for a specified constraint is returned.

We began our approach by working out the amount of subsets needed for a given input list. In the context of this problem, the order of the events in the subset does not change its assigned enjoyment value. Therefore the formula for sampling permutations was used to obtain the number of subsets, which is n^k (n is the number of choices for a given value, k is the number of values).

When an array is loaded in, a for loop is used to assign the index number for each element to a binary, for easier iteration. The binary string is then padded with zeros to the left of the value, so that each activity has a bit to be mapped to.

A for loop then iterates through the input list, using the binary, to create the current subset. For the characters that are a '1' in the binary list, their corresponding activities are appended to the current arrangement list. An if statement checks the subsets duration value against the specified duration constraint. If the subset duration values is less than the duration constraint, the check passes. Then, the current arrangement is compared against the currently stored optimal subset's enjoyment value If the current arrangement is better, then the best arrangement list is updated to the set of events in the current arrangement. Upon the loops completion, the best arrangement list is returned.

In regards to time complexity, the algorithm can be modelled as a tree, in which each layer has double the number of subsets as the previous layer (when viewed from top down). This means the formula works on and exponential time complexity for increasing input sizes. Regarding space complexity, this formula is linear, as it returns a subset of the original list, which is a new memory address. The memory of the subset returns can increase if the input size increases.[1]

What is Dynamic Programming

Dynamic programming is an approach to recursive problems in which the results of subproblems are stored and used to avoid repeating calculations and skip unnecessary steps improving the time complexity of the algorithm (Usually from exponential to polynomial).[2]

---

[1] “Space Complexity of an Algorithm - Types, Graph, Examples” Tutorial Kart.com. Accessed: Feb 11, 2026 [Online] Availible: https://www.tutorialkart.com/algorithms/space-complexity/

[2] Gregg Hogg . *Dynamic Programming - Top Down Memoization & Bottom Up Tabulation - DSA Course in Python Lecture 15*. (21 Jul 2024). Accessed Jan 25 2026 [Online Video]. Available: https://www.youtube.com/watch?v=piAlsJySUGE

Dynamic Solution Pseudocode

```
activity_count = LENGTH OF activities # Number of activities
max_weight = maximum_duration # Maximum activity weight

mat = []

# Setup the matrix
FOR row=0 TO row=activity_count + 1:
    activity_row = []

    # Fill the matrix row with zeros
    FOR column=0 TO column=max_weight + 1:
        APPEND 0 TO activity_row
    END FOR

    # Add the row to the matrix
    APPEND activity_row TO mat
END FOR

y = 0
# Loop over the rows in the matrix
FOR row IN mat:
    # Skip the first row
    IF y == 0:
        CONTINUE
    END IF

    # Get the row's focused activity
```

```
        row_activity = activities[y – 1]

        activity_joy = row_activity[“enjoyment”]
        activity_weight = row_activity[“duration”]

        FOR x=0 TO x=LENGTH OF row:
            # The value above the current square in the matrix
            joy_above = mat[y - 1][x]
            # Offset the joy above to counter the weight of the current activity
            joy_above_weight_offset = mat[y - 1][x - activity_weight]

            # If activities weight is too large then the best solution
            # is the joy above as its too heavy to add in currently
            IF x < activity_weight:
                mat[y][x] = joy_above
            ELSE:
                mat[y][x] = max(activity_joy + joy_above_weight_offset, joy_above)
            END IF
        END FOR

        # Increment the row index
        ADD 1 TO y
END FOR

best_arrangement = []
column_index = 0

# Flip the matrix upside down (reverse order)
```

```
rev_mat = REVERSE ORDER OF mat

# Start at the top right corner (best enjoyment value) and
# work backwards to figure out which activities should be included
y = 0
FOR row IN rev_mat:
        # Skip the last row
        IF y + 1 >= LENGTH OF rev_mat:
                CONTINUE
        END IF

        # Reverse the order of the row so that now the
        # best enjoyment is at location (0, 0)
        reversed_row = REVERSE ORDER OF row
        reversed_row_above = REVERSE ORDER OF rev_mat[y + 1]

        # Reverse the order of the activities to match the
        # Reversed activities order row
        reversed_activities = REVERSE ORDER OF activities

        # Get the current row activity's "weight"
        row_activity = reversed_activities[y]
        row_weight = row_activity["duration"]

        # Get the current square's value and the square above its value
        current_value = reversed_row[column_index]
        value_above = reversed_row_above[column_index]

        # If the 2 values are different then the activity is
```

```
        # part of the best arrangement
        IF current_value != value_above:
            # Add the activity to the best arrangement list
            APPEND row_activity TO best_arrangement

            # Shift the column index by the row activity's weight
            ADD row_weight TO column_index
        END IF

        # Increment the row index
        ADD 1 TO y
END FOR

RETURN best_arrangement
```

Approach and Logic

We start by initialising a n by m matrix where n is the number of different activities plus 1 ("plus 1" as the top row has no activity associated with it as no activity may be able to fit within the given time frame) and m is the maximum duration of the activities plus 1 ("plus 1" as both 0 and the maximum are included). In this matrix each of its rows maps to 1 unique activity from the activity list (apart from the top which has no activity mapped to it) and each column has a duration 1 more than the previous starting from 0 on the left most column to the maximum duration on the right most column. All values are initialised at 0.

We start one row down and one row in from the top-left corner of the matrix (as the first row remains as zeros since it has no corresponding activity as there is no corresponding activity and the first column remains zero as no activity can fit within a duration of 0). First, we check if the row's activity can fit within the current column's duration (the row activity's duration is less than or equal to the column's duration), if not then we set the current elements value to be the value of the element to its left (we can't add in the activity so no change), otherwise we set the element to be the larger value of either:

- The element to its left
- The value of the element which is on the row above and current row activity's duration steps to the left (offset is to account for the current activities duration) plus the row activity's enjoyment

(This works by maximising the enjoyment from the 2 options: include the new activity or not). We then move to the grid cell to the right (next column, same row) and perform this operation again. When the end of a row is hit, we simply move down to the next row starting once again 1 in from the first column and perform this whole process again for the next activity. This is repeated until we reach the bottom-right of the matrix (every activity has been checked).

Once all the matrix values have been set the value of the bottom-right corner of the matrix is the total enjoyment of the best activity layout within the given maximum duration. [3]

To convert the matrix into the best arrangement of activities start at the bottom-right corner. Compare the element to the one directly above it, if they are equal then the current row's activity is not in the best arrangement so move to the element above, if they are different then add the current row's activity to the best arrangement list and move up a row and shift to the element which is current row activity's duration steps to the left (This accounts for the added activities duration). Repeat this process on the next element until you reach the top row / no more elements can be added to the best arrangement list.[4]

This will be the best arrangement of activities that are within the maximum duration and maximise total enjoyment.

This is better than brute force as instead of checking every possible combination it introduces each activity one at a time and calculates the new best arrangement by

---

[3] ByteQuest *0/1 Knapsack Problem Explained Visually*. (5 Mar 2025). Accessed Jan 27 2026 [Online Video]. Available: https://www.youtube.com/watch?v=qxWu-SeAqe4

[4] WilliamFiset *0/1 Knapsack problem | Dynamic Programming*. (17 Sept 2017) . Accessed Jan 27 2026 [Online Video]. Availible: https://www.youtube.com/watch?v=cJ21moQpofY

checking whether enjoyment is increased by including the new activity in the arrangement or not using the previous best arrangement in the matrix.

Time and Space Complexity

The time complexity of the algorithm is linear with respect to the input size as the number of activities only affects the height of the matrix (the width is dependent on the maximum duration) meaning as the number of activities increases the iterations for each loop scale linearly with it.

The space complexity of the algorithm is quadratic with respect to the input size as a matrix whose size is dependent on the number of activities is created by the function (and since maximum duration can also be modelled to increase with the total number of activities) the number of rows and columns of the matrix are both dependent on the number of activities resulting in a quadratic space complexity.

4. Testing and Results

To test our algorithms, we created a testbed file called event_planner.py. This program imported a specified input file and the 2 algorithms, ran them and displayed the results. This allowed for direct comparison of their outputs and runtime, which proved useful for checking the algorithms effectiveness.

Each test was run 3 times, on the same machine. This meant the only factor affecting the runtime of an algorithm on an input was the inputs size, not differences in performance between machines. We ran each test multiple time to try and eliminate any random fluctuations in runtime that may be due to background process on our machine, which we could not control.

Results shown below:

activities	brute force time seconds	dynamic time seconds	greedy heuristics seconds

input_small	3.4499995410442355e-05	3.270000219345093e-05	1.3300000689923763e-05
input_small	3.460000455379486e-05	3.129999339580536e-05	1.209999993443489e-05
input_small	3.5899996757507325e-05	3.270000219345093e-05	1.2600000016391278e-05
average	3.499999890724818e-05	3.223333259423574e-05	1.2666666880249978e-05
input_medium	0.00398989999294281	5.950000137090683e-05	1.7500000074505806e-05
input_medium	0.0036188000068068504	5.2500002086162564e-05	1.5300000086426735e-05
input_medium	0.003907500006258488	5.5199995636940004e-05	1.4099999330937862e-05
average	0.0038387333353360497	5.573333303133646e-05	1.56333331639568e-05
input_large	57.22985329999775	0.00012550000846385957	2.4900000542402266e-05
input_large	56.414567999996244	0.00013439999520778657	2.3300000466406347e-05
input_large	54.64805209999532	0.00011760000139474868	2.199999988079071e-05
average	56.097491133329775	0.0001258333350221316	2.3400000296533105e-05

For the 5 event input file, both brute force and dynamic programming had comparable performance, with Dynamic programming being faster by 8.5%. With the averages for brute force and dynamic programming being approximately 32 and 35 microseconds respectively, both would be considered viable.

For the medium input file of 10 events, the dynamic programming performance gulf widens substantially, demonstrating its benefits compared to brute force. Dynamic programming was faster by approx. 689 %. Though both are still technically viable, the gain of dynamic programming makes it the obvious choice.

For the large input file of 25 events, the performance gulf grows even larger. This further emphasises the difference in exponential to polynomial time complexity. Dynamic programming is faster by a rate of approx. 467,500%. With an average runtime of 0.1 milliseconds

One limitation we did encounter was a lack of a definitive optimal subset for each input file . Because of this, we assumed that the result given by the brute force algorithm was correct. This made it difficult for us to independently test the algorithms' correctness for any input file apart from the first, due to its small size .

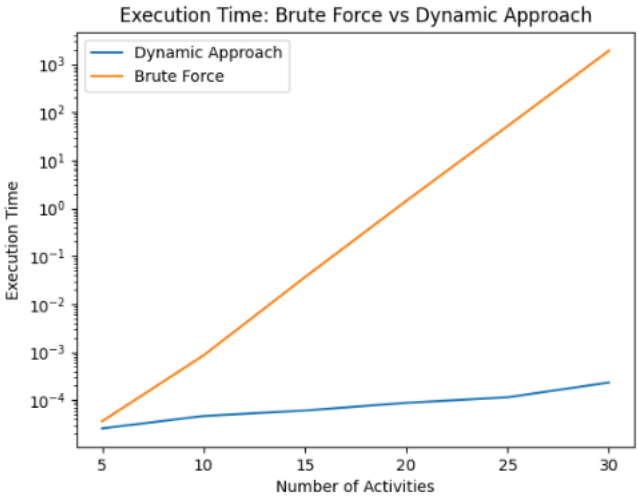
Regarding edge cases, we did create some additional test cases to see at which point the brute force would take more than 10 minutes, which we considered to be the limit of practicality. With a test case of 30 events, we recorded an execution time of 20 minutes for brute force.

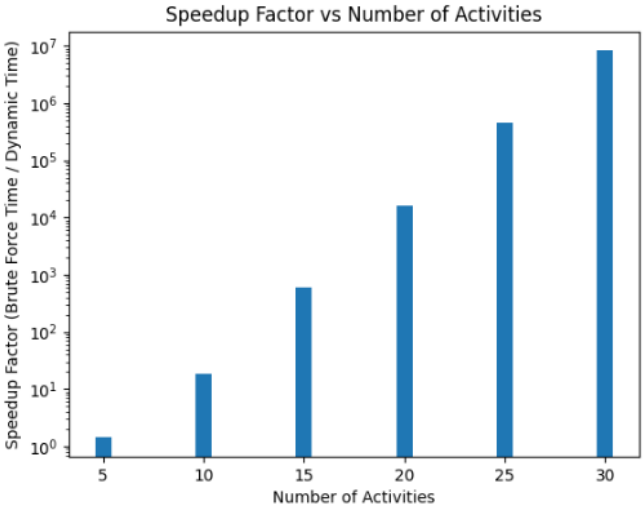
5. Extensions

Greedy Heuristic Comparison:

The greedy heuristic is an algorithm which always selects the option which gives it the most enjoyment per cost. This makes it very fast, because it does very few comparisons and grows slowly, with a worst case of O(nInclog(n)). This is shown well in the documentation above, which presents the execution times of brute force, dynamic programming, and greedy heuristic solutions. At the start they all perform at similar times on the small dataset, but by the large dataset there are massive gaps between them, with an over 400,000x increase in execution time of brute force over dynamic programming, and a further over 5x increase in execution time of dynamic programming over the greedy heuristic. However, as shown in the heuristic_comp.txt document, the greedy heuristic does not always come up with the best solution, as there is sometimes a set of activities that better fits into the allowed budget, increasing enjoyment without necessarily selecting the instructions with the best ratio of enjoyment to cost. In all cases but one in the heuristic_comp.txt document, the greedy heuristic comes up with the same solution as both dynamic and brute force, and in the one case where it doesn't, the difference in enjoyment is only 20 (or just over 3%), which is a very minute difference when compared with the vast increase in run time, even over dynamic programming.

Performance Analysis





<u>At what value of n does brute force become impractical (e.g., taking more than 10 minutes)?</u>

In the line graph, we observe a highly linear relationship between execution time and number of activities (n).

Therefore, we can calculate the rate that execution time increases, as n (the number of activities in the list) increases, using the formula:

$$time = 10^{0.314n-6.13}$$

This will allow us to extrapolate the data and allow us to find the hypothetical execution time for any value of n.

Calculation: Finding n where time > 10 minutes

- 10 minutes = 600,000 ms
- log(600,000) = (0.314)n – 6.13
- (5.778 + 6.13) / 0.314 = n
- = 37.92…

Hence, brute force begins to be impractical at around 38 activities, as at this value of n, the program would take greater than 10 minutes to execute.

How does the dynamic programming execution time grow as n increases?

By observing the shape of the line graph (linear) we can derive a formula for the relationship between execution time and n for the dynamic approach program.

$$time = 10^{0.0615n-5.4768}$$

6. Generative AI Statement

AI was not used in any portion of this coursework.

7. References

"Space Complexity of an Algorithm - Types, Graph, Examples" Tutorial Kart.com. Accessed: Feb 11, 2026 [Online] Available: https://www.tutorialkart.com/algorithms/space-complexity/

Gregg Hogg . *Dynamic Programming - Top Down Memoization & Bottom Up Tabulation - DSA Course in Python Lecture 15*. (21 Jul 2024). Accessed Jan 25 2026 [Online Video]. Available: https://www.youtube.com/watch?v=piAlsJySUGE

ByteQuest *0/1 Knapsack Problem Explained Visually*. (5 Mar 2025). Accessed Jan 27 2026 [Online Video]. Available: https://www.youtube.com/watch?v=qxWu-SeAqe4

WilliamFiset *0/1 Knapsack problem | Dynamic Programming*. (17 Sept 2017) . Accessed Jan 27 2026 [Online Video]. Available: https://www.youtube.com/watch?v=cJ21moQpofY